

Assignment 2: Coding Basics

Tracy Che

OVERVIEW

This exercise accompanies the lessons/labs in Environmental Data Explorations on reproducibility and coding basics.

Directions

1. Rename this file `<FirstLast>_A02_CodingBasics.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. After Knitting, submit the completed exercise (PDF file) to Canvas.

Crafting Reports - Part 1

Import packages and data, suppressing messages

Set the following R code chunk so that it runs when knit, but no messages, errors, or any output is shown. The code itself, however, should be displayed.

```
#Import libraries  
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --  
## v dplyr      1.1.4      v readr      2.1.5  
## v forcats    1.0.0      v stringr   1.5.1  
## v ggplot2    3.5.1      v tibble    3.2.1  
## v lubridate  1.9.3      v tidyr     1.3.1  
## v purrr      1.0.2  
## -- Conflicts ----- tidyverse_conflicts() --  
## x dplyr::filter() masks stats::filter()  
## x dplyr::lag()     masks stats::lag()  
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(knitr)
```

Coding Basics - Part 2

1. Generate a sequence of numbers from one to 55, increasing by fives. Assign this sequence a name.
2. Compute the mean and median of this sequence.
3. Ask R to determine whether the mean is greater than the median.
4. Insert comments in your code to describe what you are doing.

```
#1. create a sequence containing the numbers and display
sequence_1 <- seq(from = 1, to = 55, by = 5)
sequence_1
```

```
## [1] 1 6 11 16 21 26 31 36 41 46 51
```

```
#2. calculating mean and median, assigning them to variable names, and display
mean_value <- mean(sequence_1)
median_value <- median(sequence_1)

mean_value
```

```
## [1] 26
```

```
median_value
```

```
## [1] 26
```

```
#3. if mean is greater than median, it will return true, otherwise false
mean_value > median_value
```

```
## [1] FALSE
```

Coding Basics - Part 3

5. Create three vectors, each with four components, consisting of (a) student names, (b) test scores, and (c) whether they are on scholarship or not (TRUE or FALSE).
6. Label each vector with a comment on what type of vector it is.
7. Combine each of the vectors into a data frame. Assign the data frame an informative name.
8. Label the columns of your data frame with informative titles.

```
#5 and 6
#Character vector
names <- c('Tracy', 'Mary', 'Andrew', 'Scott')

#numerical vector
scores <- c(96, 99, 98, 95)

#logical vector
```

```
scholarship_status <- c(FALSE, TRUE, TRUE, TRUE)

#7 assign to dataframe and assign names
student_data <- data.frame(names, scores, scholarship_status)

#8 assign the column names
colnames(student_data) <- c("Student_Name", "Test_Score", "Scholarship_Status")

student_data
```

```
##   Student_Name Test_Score Scholarship_Status
## 1      Tracy      96          FALSE
## 2      Mary      99           TRUE
## 3    Andrew      98           TRUE
## 4     Scott      95           TRUE
```

9. QUESTION: How is this data frame different from a matrix?

Answer: data frame contains different types of data, such as character and numbers, while in a matrix all the data should be same type

10. Create a function with one input. In this function, use `if...else` to evaluate the value of the input: if it is greater than 50, print the word "Pass"; otherwise print the word "Fail".
11. Create a second function that does the exact same thing as the previous one but uses `ifelse()` instead of `if...else`.
12. Run both functions using the value 52.5 as the input
13. Run both functions using the **vector** of student test scores you created as the input. (Only one will work properly...)

```
#10. Create a function using if...else
test_1 <- function(x) {
  if (x > 50) {
    print("Pass")
  } else {
    print("Fail")
  }
}

#11. Create a function using ifelse()

test_2 <- function(x) {
  print(ifelse(x > 50, "Pass", "Fail"))
}

#12a. Run the first function with the value 52.5

test_1(52.5)
```

```
## [1] "Pass"
```

#12b. Run the second function with the value 52.5

```
test_2(52.5)
```

```
## [1] "Pass"
```

#13a. Run the first function with the vector of test scores

```
#test_1(scores)
```

#13b. Run the second function with the vector of test scores

```
test_2(scores)
```

```
## [1] "Pass" "Pass" "Pass" "Pass"
```

14. QUESTION: Which option of `if...else` vs. `ifelse` worked? Why? (Hint: search the web for “R vectorization”)

Answer: only `ifelse` function worked. It is because that the `if...else` function including the `if (x > 50)` statement can only check one condition at a time, but a vector gives the function multiple data.

Crafting reports - Part 4

Knit and submit.

Add a table of contents to your document and knit to a PDF. Submit your PDF to Canvas, but also be sure to commit and push your Rmd file used to create this knit document to GitHub. In the section below, add a link to your GitHub repository.

Github link

Answer: https://github.com/tracy-siyu-che/EDE_R_Lab_F26/tree/main/Assignments

NOTE Before knitting, you’ll need to comment out the call to the function in Q13 that does not work. (A document can’t knit if the code it contains causes an error!)